

Authentication

OSTEP 54

Sam Ogden

Updated 2026-06-16 15:13

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Lecture Objectives

- After this lecture, you should be able to:
- Explain three main authentication methods
- Consider the challenges of each method

Some definitions before we begin

- **Principal:** who or what is asking for permission
- **Object:** the resource being asked for
- **Credential:** tracking of permissions granted

How to assign credentials?

- We need to figure out who to let run
- And what permissions they should have
- Authentication only matters if the OS can tie identity to access



Main ways to check

Authentication usually comes from one of three kinds of evidence

- 1. What you know**
- 2. What you have**
- 3. What you are**

What do you know?

Passwords

What do you know?

Before we trust passwords, we need to ask:

- What makes a good secret?
- How do we store secrets?
- How do we check secrets?
- How do we protect secrets?

Password entropy

UNCOMMON (NON-GIBBERISH) BASE WORD

ORDER UNKNOWN

Tr0ub4dor &3

CAPS? COMMON SUBSTITUTIONS NUMERAL PUNCTUATION

(YOU CAN ADD A FEW MORE BITS TO ACCOUNT FOR THE FACT THAT THIS IS ONLY ONE OF A FEW COMMON FORMATS.)

~28 BITS OF ENTROPY

$2^{28} = 3 \text{ DAYS AT } 1000 \text{ GUESSES/SEC}$

(PLAUSIBLE ATTACK ON A WEAK REMOTE WEB SERVICE. YES, CRACKING A STOLEN HASH IS FASTER, BUT IT'S NOT WHAT THE AVERAGE USER SHOULD WORRY ABOUT.)

DIFFICULTY TO GUESS: EASY

WAS IT TROMBONE? NO, TROUBADOR. AND ONE OF THE 0s WAS A ZERO?

AND THERE WAS SOME SYMBOL...

DIFFICULTY TO REMEMBER: HARD

correct horse battery staple

FOUR RANDOM COMMON WORDS

~44 BITS OF ENTROPY

$2^{44} = 550 \text{ YEARS AT } 1000 \text{ GUESSES/SEC}$

DIFFICULTY TO GUESS: HARD

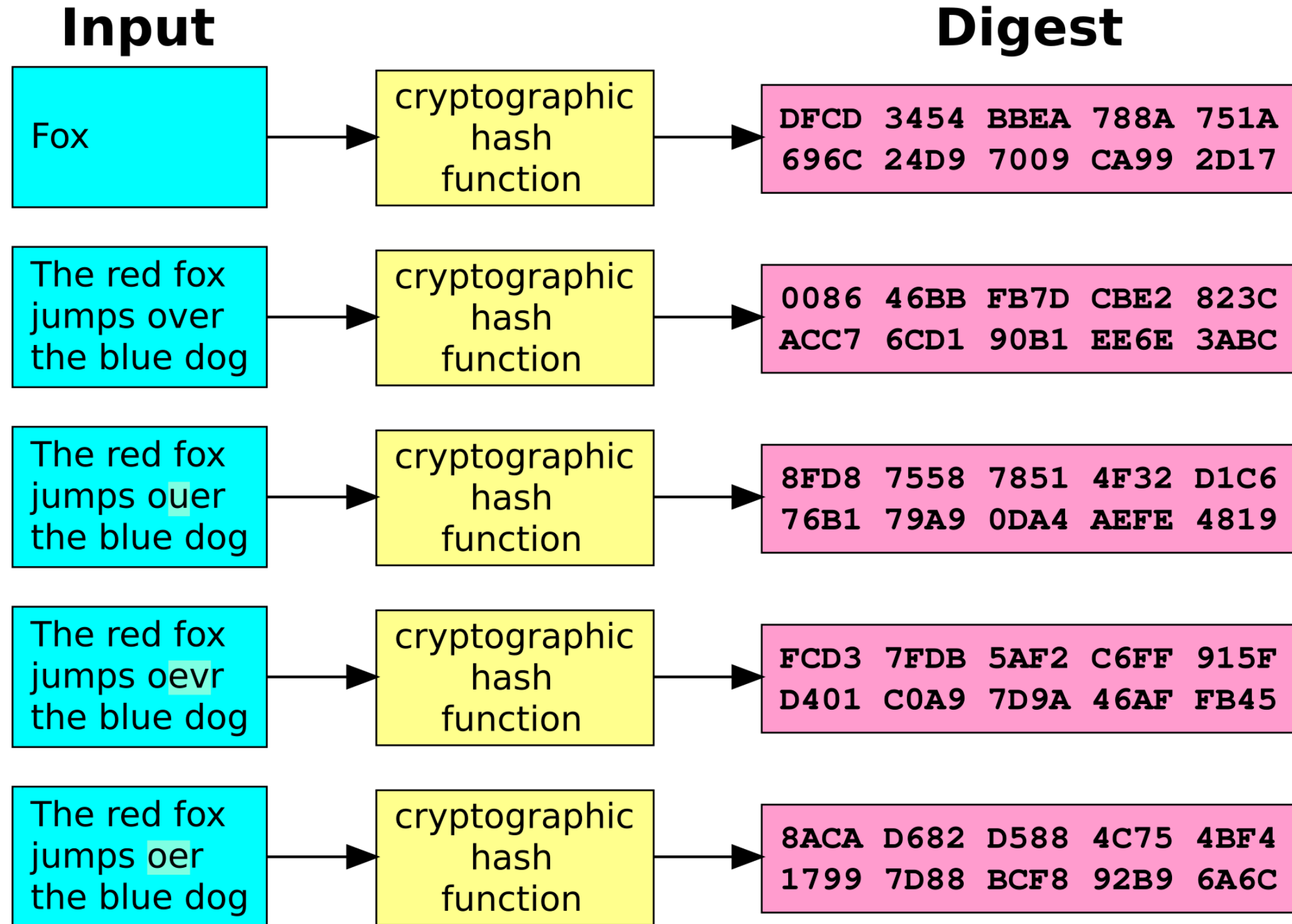
THAT'S A BATTERY STAPLE.

CORRECT!

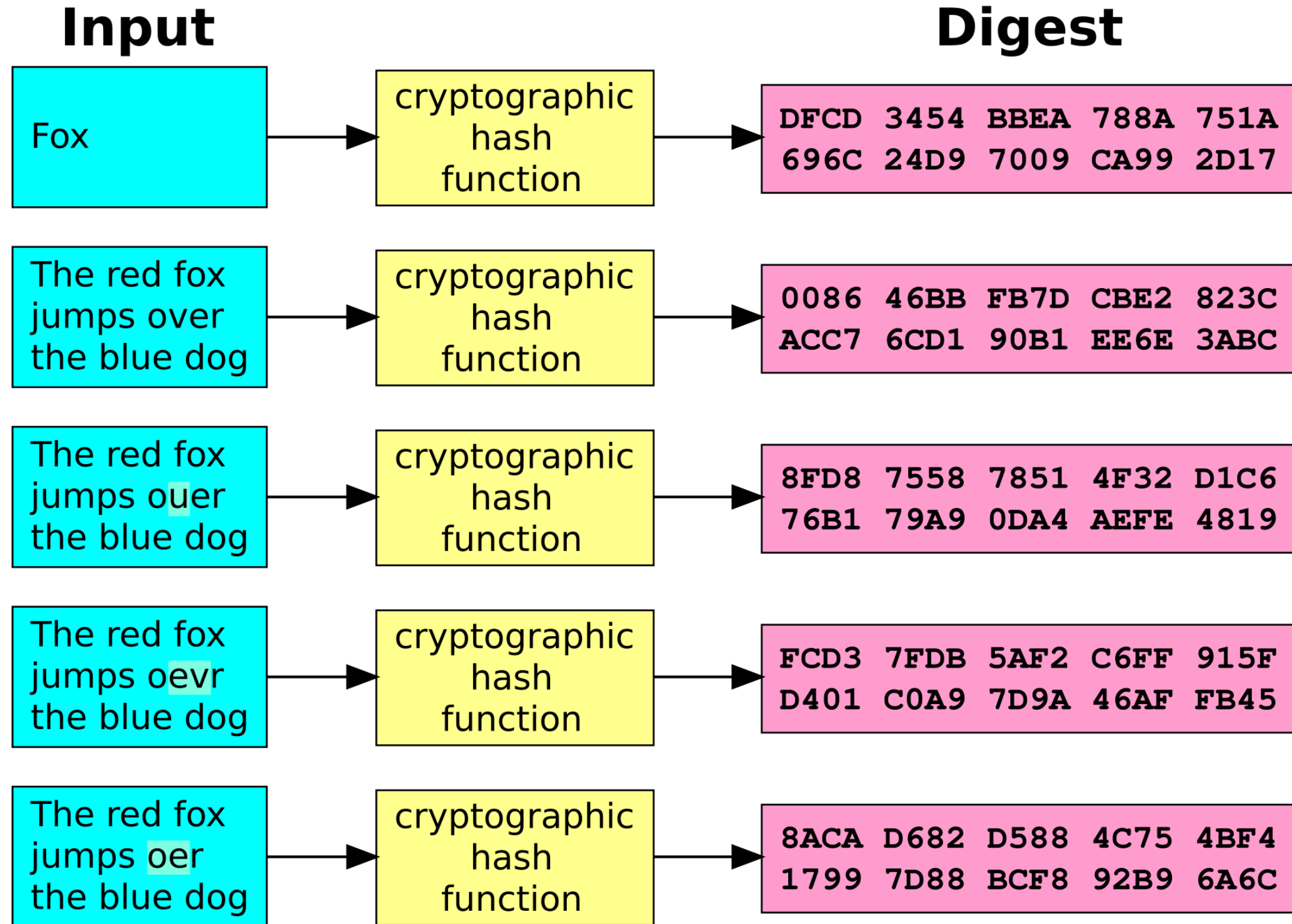
DIFFICULTY TO REMEMBER: YOU'VE ALREADY MEMORIZED IT

THROUGH 20 YEARS OF EFFORT, WE'VE SUCCESSFULLY TRAINED EVERYONE TO USE PASSWORDS THAT ARE HARD FOR HUMANS TO REMEMBER, BUT EASY FOR COMPUTERS TO GUESS.

How do we store them?



How do we check them?



How do we protect secrets?

Say somebody manages to get our password file....

If it's hashed, do we have a problem?

YES! They can try all the hashes!

So how do we fix this?

Time it takes a hacker to brute force your password in 2025

Hardware: 12 x RTX 5090 | Password hash: bcrypt (10)

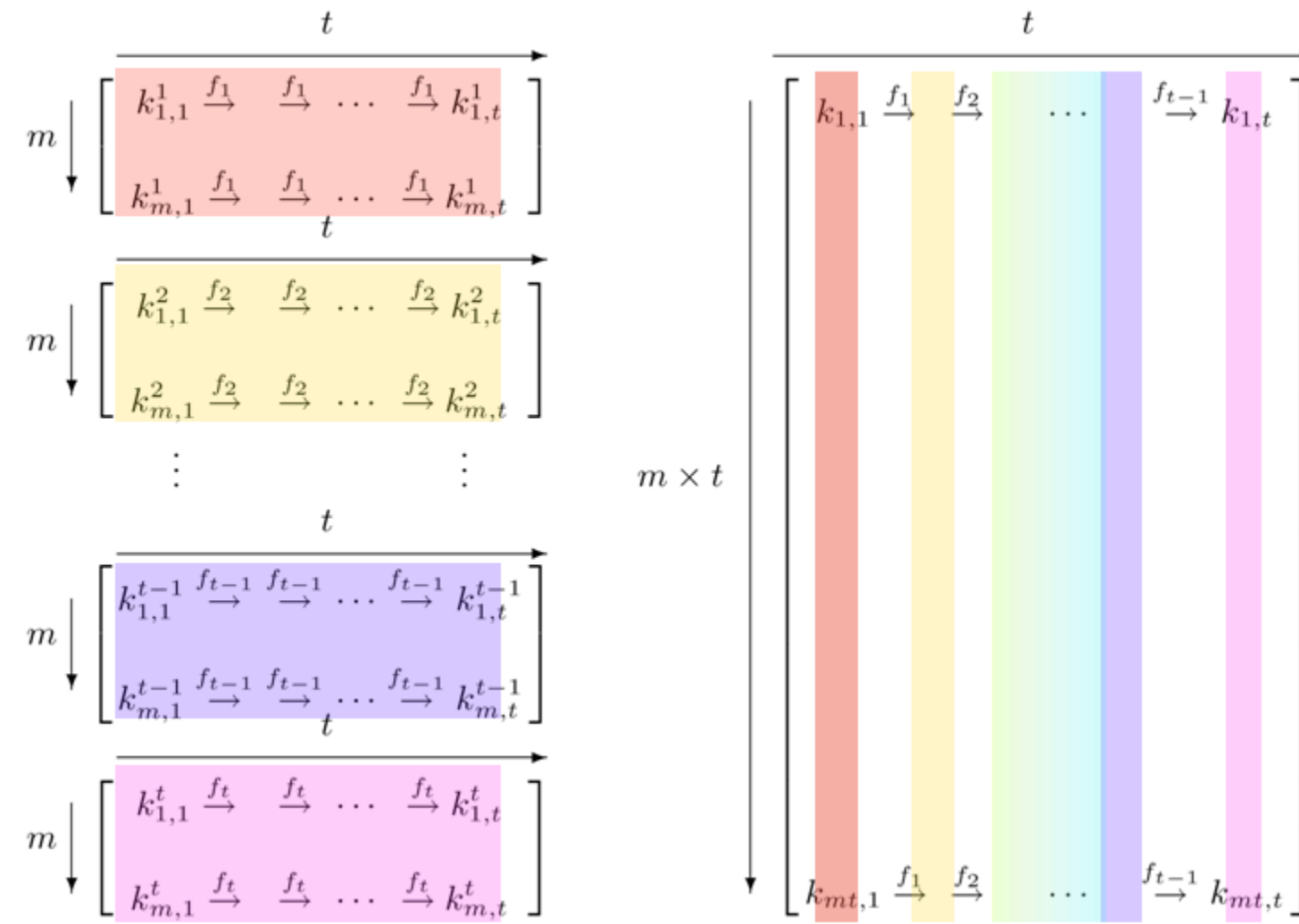
Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	57 minutes	2 hours	4 hours
6	Instantly	46 minutes	2 days	6 days	2 weeks
7	Instantly	20 hours	4 months	1 year	2 years
8	Instantly	3 weeks	15 years	62 years	164 years
9	2 hours	2 years	791 years	3k years	11k years
10	1 day	40 years	41k years	238k years	803k years
11	1 weeks	1k years	2m years	14m years	56m years
12	3 months	27k years	111m years	917m years	3bn years
13	3 years	705k years	5bn years	56bn years	275bn years
14	28 years	18m years	300bn years	3tn years	19tn years
15	284 years	477m years	15tn years	218tn years	1qd years
16	2k years	12bn years	812tn years	13qd years	94qd years
17	28k years	322bn years	42qd years	840qd years	6qn years
18	284k years	8tn years	2qn years	52qn years	463qn years



Hive Systems

Read more and download at hivesystems.com/password

Time to crack a password of a given length



Rainbow Table illustration presented at Crypto 2003

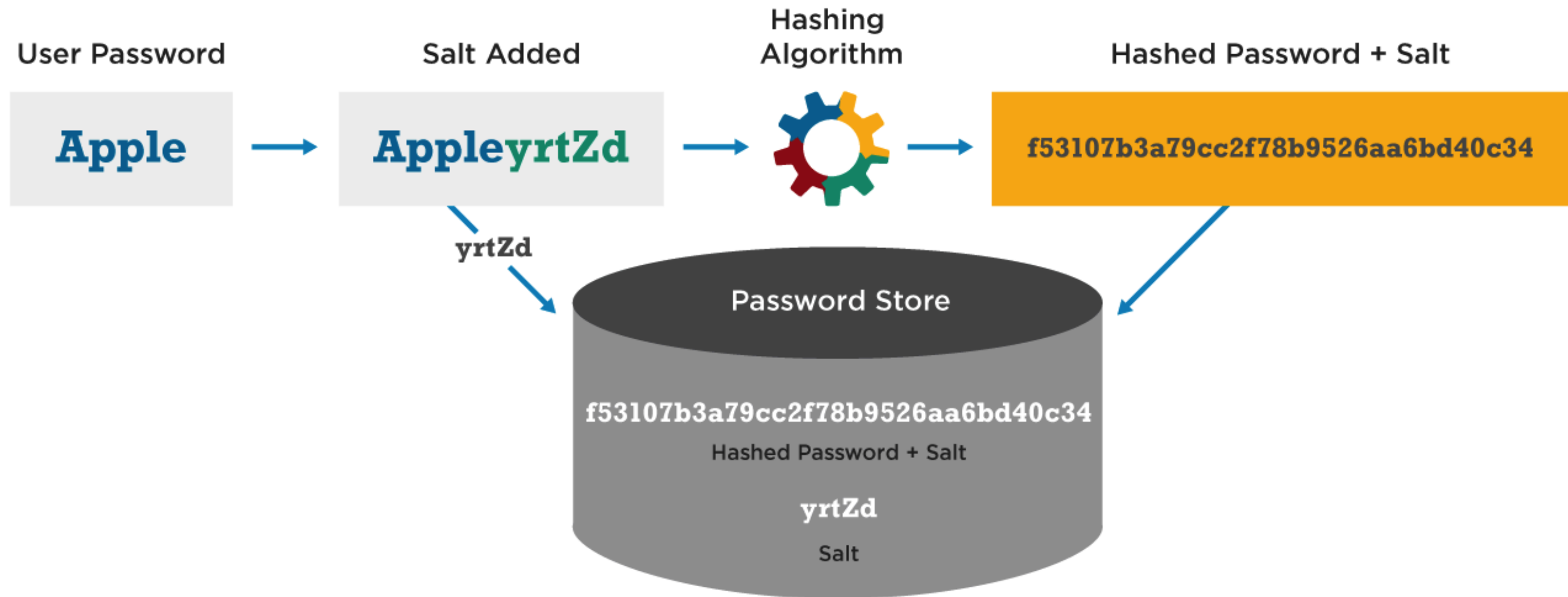
We add a salt



- A **salt** is a little extra information added to every password
 - Effectively makes it longer
- Each user has their own **salt**
 - It doesn't need to be private
- They effectively remove the benefits of precomputation (aka rainbow tables)

How does *salting* work?

Password Hash Salting



What do you know? Key Takeaways

- Use an expertly designed hashing function
- Protect your password file
- Always add salt to the hash
- Passwords that people can remember are better than ones they write down

What do you have?

Cards and Keys

What you have: What is it?

- Authentication based on having a specific thing
 - A concert ticket
 - A lanyard
 - Your physical ATM card
 - Your cell phone
 - A security dongle

What problems are there?

- What happens if you lose your device?
- What happens if somebody steals your device?
- How do you prove you're still you?
- Can somebody else use it?
- How often does it update?
- Will people use it?

What you are

Biometrics

What are biometrics?

- Measurements of what makes you you
- Retinal scan (Mission Impossible)
- Palm print (Whole Foods)
- Thumb print (iPhone SE)

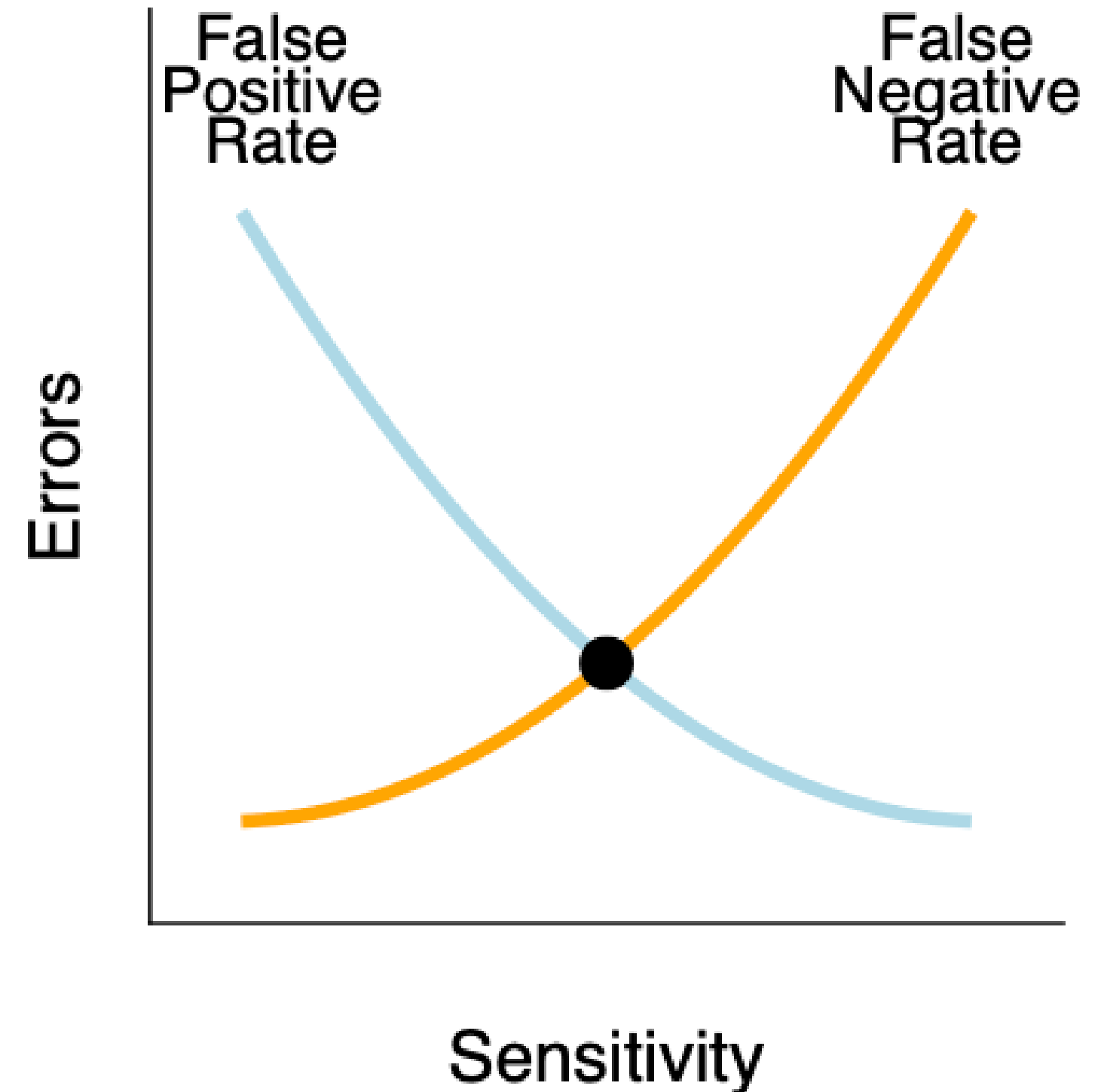
Biometrics Challenges

Are these the same cat?



Sensitivity

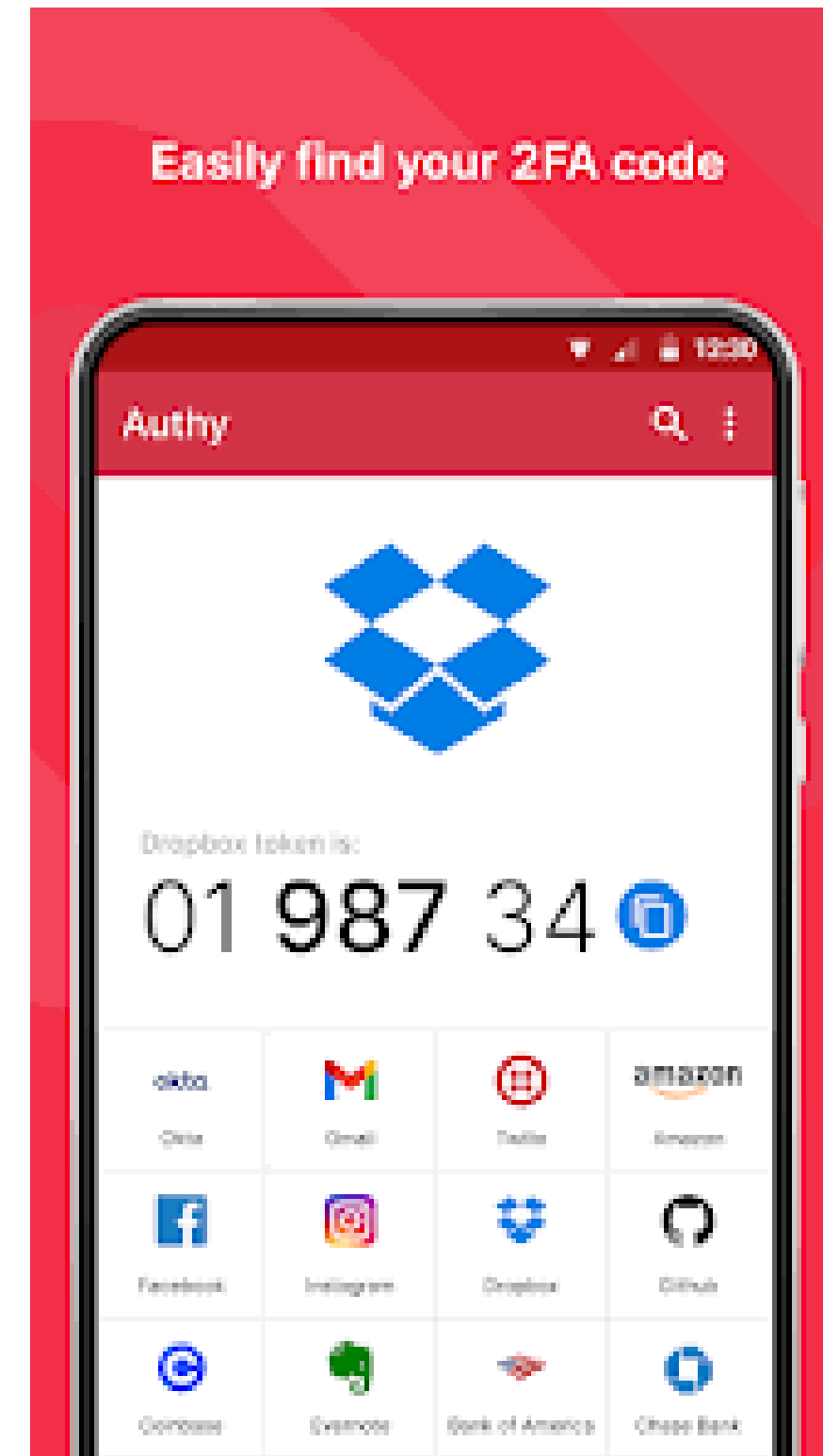
- We need to strike a balance between being too relaxed and too stringent
- Context matters!
 - How mission-critical is this?
 - What is our mission?



How can we make it better?

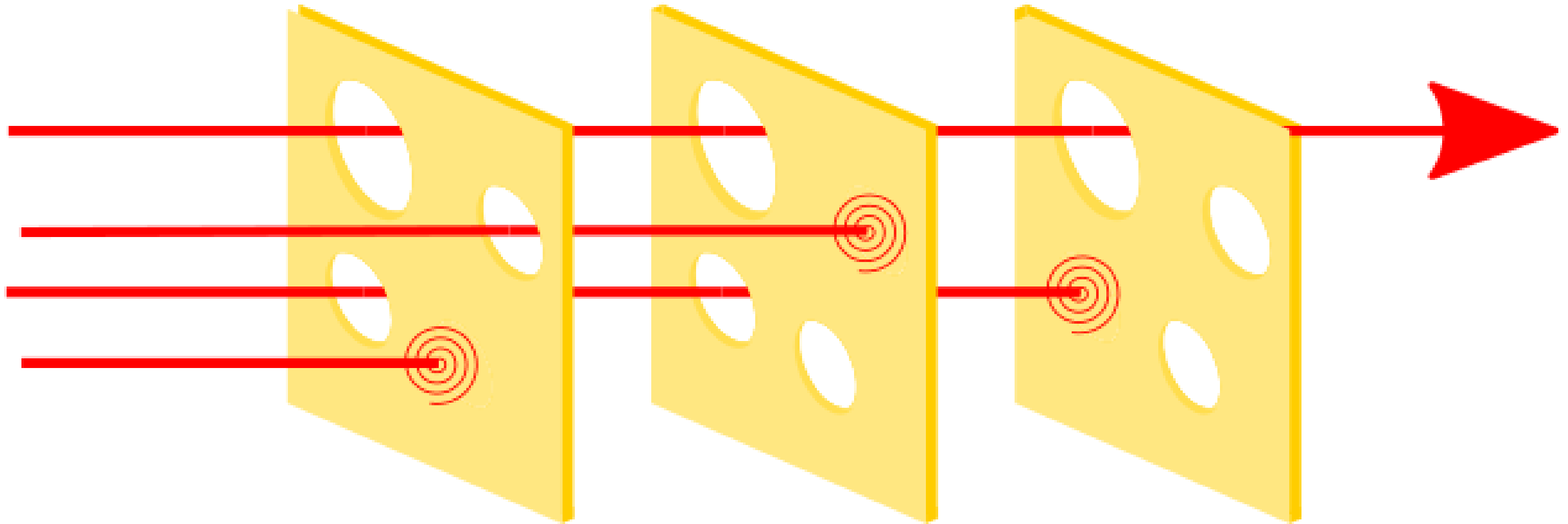
Authentication is stronger when we combine things that fail in different ways.

- Password + phone
- Multi-Factor Authentication



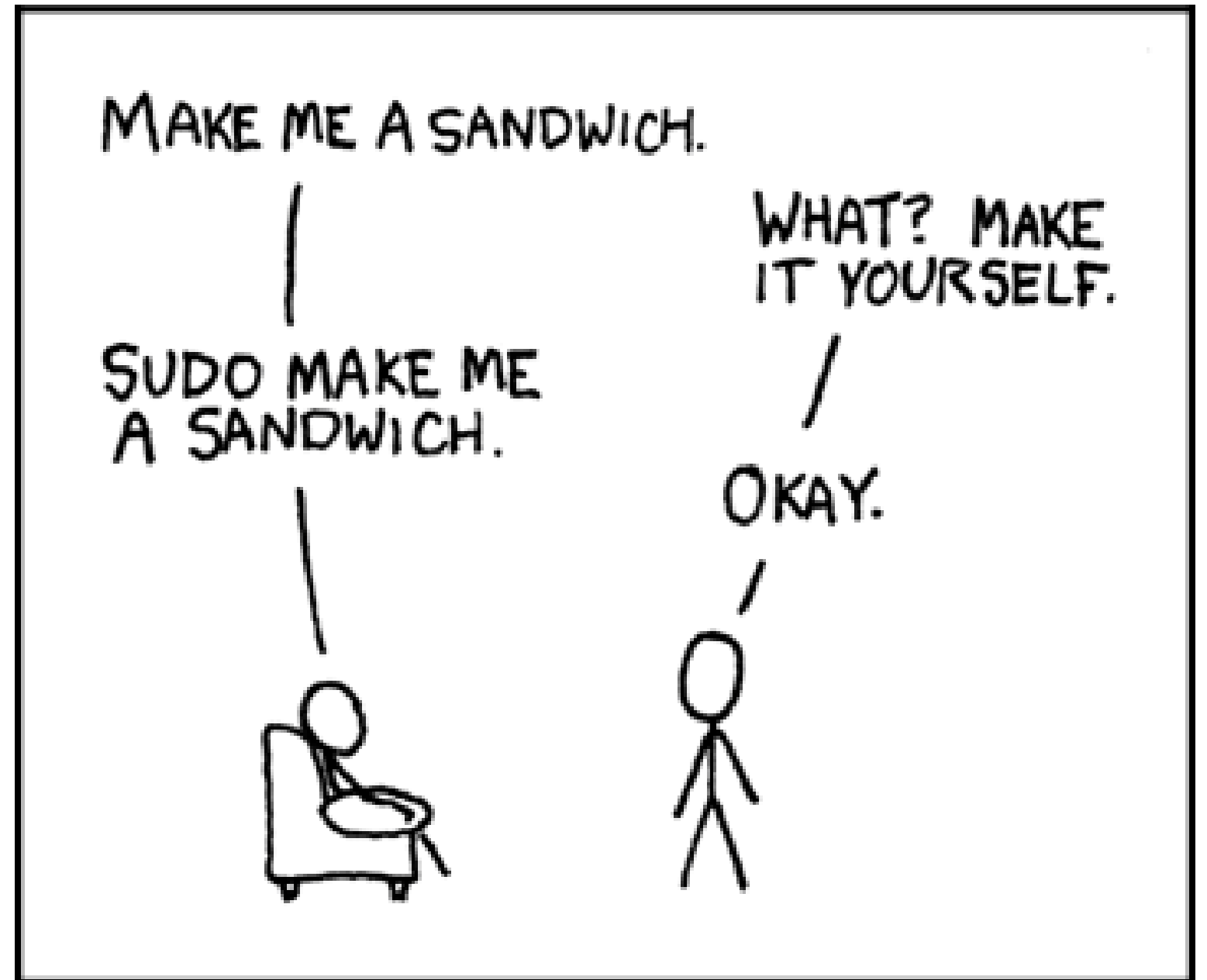
How to design a system

- The swiss cheese model says add lots of layers to help protect yourself
- If something makes it through one layer, stop it before the next



How to authenticate the system?

- Sometimes you want to run tasks not as yourself
- And you don't want to sign in
- How do we do this?



Summary

- Three main authentication methods:
 - what you know
 - what you have
 - what you are
- These work best when combined
- Need to balance strictness and usability